PRODUCT & TECHNICAL HANDBOOK / VERSION 0.1.0

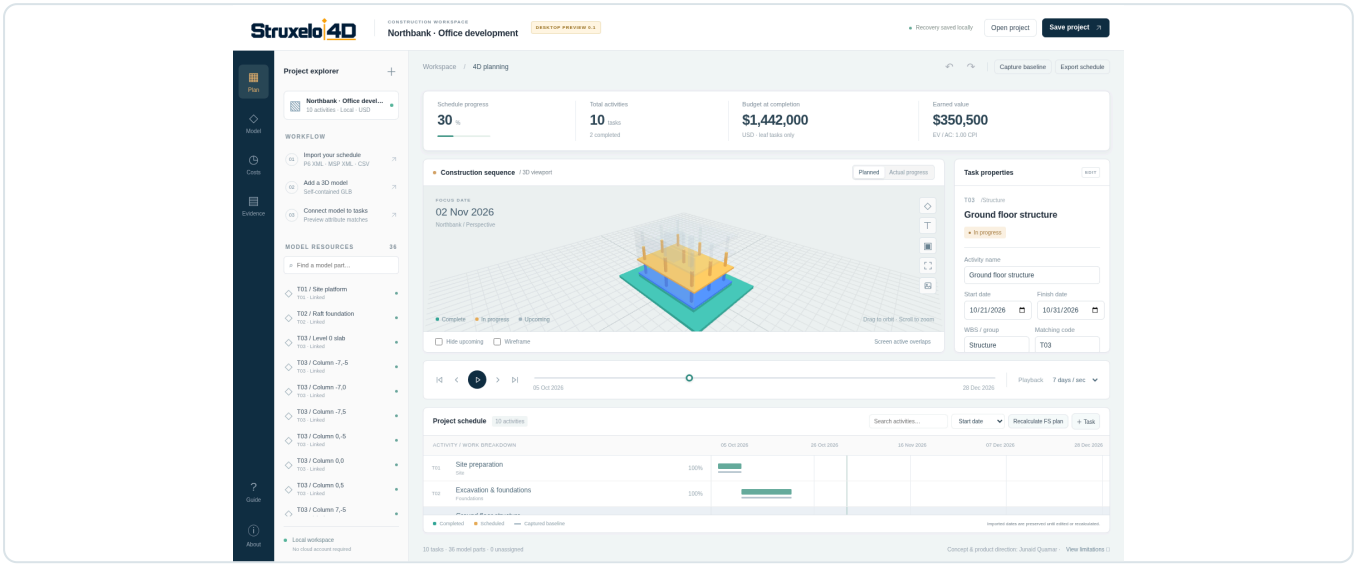
Plan the sequence. See the construction.

Windows desktop construction planning, model-linked schedules and transparent project controls.

CONCEPT & PRODUCT DIRECTION

Junaid Quamar

Senior Planning Engineer



Independent prototype. Native SYNCHRO file compatibility and full feature parity are not established. Windows runtime acceptance remains required.

Prepared 26 October 2026 · Software implementation assisted by OpenAI Codex
Includes installation, workflow, data formats, architecture, evidence, review decisions and validation.

HANDBOOK

Contents

Overview

Installation

User guide

Data formats

Architecture

Research and evidence

Feature matrix

Review decisions

Brand identity

Validation

Credits

Third-party notices

Struxelo 4D

Concept and product direction: Junaid Quamar, Senior Planning Engineer.

Struxelo 4D is an independent Windows desktop prototype for connecting construction activities to 3D model parts. It includes schedule editing, construction playback, baseline dates, physical progress and cost metrics. The renderer can also run locally in a browser for development and testing.

Version 0.1.0 is not a complete or verified replacement for SYNCHRO 4D. The current implementation was informed by public documentation, public user feedback and static inspection of Bentley's public training archive using REA. A running SYNCHRO installation was not available for differential testing. Native SP compatibility is not implemented.

Download/read the illustrated handbook · Standalone HTML handbook

Start on Windows

1. Extract the entire `Struxelo-4D-0.1.0-win-x64.zip` archive into a writable local folder.
2. Run `Struxelo4D.exe` inside that folder. Keep its accompanying files and directories together.
3. Use the demonstration project to explore selection, timeline playback, task editing and costs.
4. Save a `.struxelo` project file before replacing the demo with your own data.

The package is unsigned. Windows execution, native file dialogs and GPU behavior require validation on an actual Windows machine. See installation.

Documentation

Document	Purpose
User guide	End-to-end planning workflow, imports, playback, assignments and recovery
Architecture	Electron boundaries, renderer, model lifecycle and scheduling semantics
Data formats	Project schema, supported XML/CSV fields and import limitations
Research and sources	Official sources, REA observations, download access and uncertainty
Feature matrix	Implemented, partial and missing capabilities
Review decisions	Which public suggestions were adopted and why
Validation	Executed checks and Windows acceptance checklist
Credits	Product direction, software assistance and third-party attribution

Develop from source

Use Node.js 22.12+ or a supported current Node.js 24 release, npm and Python 3. The Windows package includes Electron and does not require users to install Node.js.

```
npm ci
npm run dev
# In another terminal, restore the public reference fixtures before tests:
npm run fixtures:download
npm test
npm run package:windows
```

The two real XML fixture tests require the official training archive in `research/samples`; see the research document for the source and digest. The packaging script uses Python’s standard ZIP support and downloads the pinned Electron Windows x64 runtime. It does not modify system Node.js or install analysis engines.

Main limitations

- Whole-date scheduling; no working calendars, constraint solver or resource leveling.
- P6 and MSP imports preserve selected fields; they do not reproduce source-application scheduling semantics.
- GLB models only. No direct IFC, RVT, DWF/DWFX or SP/SPX geometry decoder.
- Local single-user workspace. No Bentley cloud integration or multi-user synchronization.
- Approximate bounding-box screening, not exact solid clash detection.
- No Iray renderer, video export, equipment path editor or construction subdivision.

Third-party components retain their licenses. This project is independent of Bentley Systems. See third-party notices.

Installation and operation

Concept and product direction: **Junaid Quamar, Senior Planning Engineer**.

Windows distribution

Target: Windows 10 or Windows 11, x64. A working Chromium-compatible graphics driver is needed for the 3D view. The desktop window has a minimum size of 1050 × 720; 1440 × 900 or larger is more comfortable.

The deliverable is a portable directory in a ZIP, not an installer. Extract all files, then start `Struxelo4D.exe`. Moving only the executable will break Electron's runtime dependencies. The runtime is based on the official Electron distribution. Its icon and product metadata are customized for Struxelo 4D; it is unsigned.

Project files can be saved to a location chosen through the native Save dialog. Automatic recovery uses Chromium local storage in the application's user-data profile. For Windows, Electron normally places that profile below `%APPDATA%\Struxelo 4D`. The app does not connect to a project server.

The bundled example is an independent demonstration, not Bentley's training building. Start with it before importing production information. Treat this release as a prototype until the Windows acceptance checklist and representative project checks have been completed.

Local development

From the source directory:

```
npm ci
npm run dev -- --port 5173
```

Open `http://localhost:5173`. The development renderer uses browser file inputs and downloads; these differ from native Windows dialogs. Stop the development server with Ctrl+C.

Build production assets:

```
npm run build
```

Package Windows x64:

```
npm run package:windows
```

The packaging script requires Python 3, network access to Electron's official release distribution and enough free space for the extracted runtime and ZIP. In a proxy-based environment, keep the provided proxy and CA configuration; set `ELECTRON_GET_USE_PROXY=1` if @electron/get requires it. Do not disable certificate verification.

The resulting folder and ZIP are in `release/`. The script downloads a pinned runtime, verifies its published checksum, places the built application in `resources/app`, copies the manuals and keeps runtime licenses.

REA installation

REA 6.8.0 was built from GitHub commit `0211a07ec5c43b2f8cbdbcd80f3f3619773a7b72` in `/workspace/rea`. The public launcher is:

```
/workspace/bin/rea --version
/workspace/bin/rea mcp doctor --json
```

Its pinned Node.js and npm are isolated under `/workspace/rea-toolchain`. No native decompiler was installed. REA's CLI works independently of agent registration; a successful adapter test does not mean REA tools have been injected into this already-running chat session.

Troubleshooting

Symptom	Action
The app does not start after extraction	Confirm the complete runtime folder was extracted. Test on a Windows x64 machine.
3D is unavailable	Update the graphics driver and verify hardware acceleration. Schedule editing can still be used.
Recovery cannot be saved	Save a project file. The local storage quota may be exhausted, especially with embedded GLB data.
A GLB fails	Use an uncompressed, self-contained binary glTF. External buffers/textures, skinned meshes and decoder-dependent assets are outside the supported subset.
P6/XML import reports omissions	Read the import report. Original dates are preserved; unresolved relationships are retained and block recalculation.
Task changes are rejected	Check ISO dates, negative costs, dependency IDs and circular links. No invalid edit is committed.

User guide

Struxelo 4D 0.1.0
Concept and product direction: **Junaid Quamar, Senior Planning Engineer**.

Workspace

The left rail selects Plan, Model, Costs and Evidence. The project explorer contains a three-step workflow and searchable model resources. The main workspace holds summary metrics, the 3D viewport, a task inspector, playback controls and a schedule or information panel.

Plan and Model currently share the same model-and-schedule workspace; Model does not expose a separate geometry authoring suite. Costs changes the lower panel to a cost table. Evidence explains what was observed and what remains unsupported. Guide provides an in-app introduction; About displays project credits.

The opening Northbank project has ten activities and a simple independently generated building. It is demonstration data, not a calibrated construction plan. Select a task row to inspect it. Select a model part in the explorer or viewport to highlight it; clicking a linked part in the viewport also selects its assigned task.

Bring in a schedule

1. Save the current workspace if needed.
2. Choose **Import your schedule**.
3. Select a P6 XML, Microsoft Project XML or supported CSV.
4. Confirm replacing the current project. Schedule imports clear existing geometry and Undo history.
5. Read the import report. It describes fields that were not interpreted and any retained unresolved relationships.

Dates are imported as whole dates. The program does not immediately recalculate them. Summary rows are preserved for MSP imports; P6 activities carry their WBS identifier or code. This prototype does not evaluate working calendars, constraints or source scheduling rules.

For CSV, provide `id`, `name`, `start`, and `finish` headers. Optional headers include `wbs`, `code`, `progress`, `budget` and `actualCost`. Use `YYYY-MM-DD` dates and numeric values without currency symbols. Quote names containing commas.

```
id,name,wbs,code,start,finish,progress,budget,actualCost
A100,Foundations,Civil,FOUND,2026-11-01,2026-11-12,25,120000,28000
A110,Steel frame,Structure,STEEL,2026-11-12,2026-11-26,0,200000,0
```

Add and link a model

Choose **Add a 3D model** and select a self-contained `.glb` file up to 25 MB. Importing replaces the current geometry and its assignments, while retaining schedule tasks. The prototype uses the model's native coordinate units; it does not infer a CAD unit system.

GLB animations, skinning, compressed geometry extensions and external resources are not supported workflows. Imported parts use standardized display materials so that construction status is legible. Original material fidelity is not preserved.

To assign manually:

1. Select a model resource in the explorer or viewport.
2. Select its intended task in the schedule.
3. In Task properties, scroll to **Linked model parts**.
4. Click **Link selected part**. Unlink removes that assignment.

Each part has one task assignment in this prototype. Many parts may reference the same task. The resource-to-task graph does not reproduce SYNCHRO's complete resource hierarchy or many-to-many appearance semantics.

Automatic assignment uses a task's **Matching code**. A code of at least three characters must appear in a part name, ignoring case. Only unassigned parts with exactly one matching task are proposed. Review the preview before applying. Ambiguous matches remain unassigned.

Review construction

- Drag the timeline to choose the focus date.
- Use previous/next to step exactly one calendar day.
- Use beginning/end to jump to the project bounds.
- Choose a playback rate of 1, 7 or 30 days per second, then Play.
- Planned mode derives status from task dates. Actual progress mode uses recorded task percentages and is not a historical status reconstruction.
- Hide upcoming removes future parts in Planned mode and zero-progress parts in Actual progress mode.
- Orbit by dragging; zoom with the mouse wheel. The toolbar provides isometric, top, front and fit views.

Colors identify complete, active and upcoming parts; blue indicates selection. A milestone changes state on its date. A zero-duration task has no active interval between dates.

Screen active overlaps compares world-space axis-aligned mesh bounding boxes at one focus date. Positive overlaps are candidates for review, not exact solid intersections or certified clash results. Touching faces are excluded by a small numerical tolerance. Screen at most 1,000 active parts at once.

Use **Export PNG** for a viewport image. It contains the rendered scene, not a full report with DOM labels or the schedule. Video export is not implemented.

Edit activities and record progress

Select a task. Edit its name, WBS, matching code, dates, physical progress, budget or actual cost, then **Apply task changes**. Invalid dates, negative costs, progress outside 0–100, missing dependency IDs and dependency cycles are rejected.

Dependencies use comma-separated `ID:TYPE:lag` entries, for example `A100:FS:0, A105:FF:2`. Lag is in calendar days in the internal project. FS, SS, FF and SF links can be stored, but only FS links can be recalculated by the limited prototype engine.

Recalculate FS plan is explicit and requires confirmation. It supports flat tasks, calendar days, fixed current date-span durations and FS dependencies. The earliest task date becomes the origin. It does not preserve independently imposed task start constraints. Summary rows, other link types or retained unresolved import relationships block recalculation.

Capture baseline stores the current start/finish dates and replaces the prior baseline. Gray lines appear beneath current Gantt bars. Capture can be undone. The baseline is a date snapshot, not a complete historical schedule or a separate cost budget.

Costs

Values are expressed as USD; currency selection is not implemented. Summary rows are excluded to prevent double counting.

- Budget at completion: sum of leaf-task budgets.
- Earned value: sum of each leaf-task budget × entered physical progress / 100.
- Actual cost: sum of entered leaf-task actual costs.
- Planned value: budget spread linearly across baseline dates, or current dates when no baseline exists, evaluated at the focus date.
- CPI: earned / actual cost, or unavailable when actual cost is zero.
- Cost variance: earned minus actual cost.

Planned value is a simplified linear approximation. These calculations do not reproduce resource curves, contractual valuation rules or all P6/MSP earned-value settings. Headline progress is the unweighted mean of leaf-task percentages, not a budget- or quantity-weighted project percentage.

Saving, recovery and sharing

Save project writes a `.struxelo` JSON file. Embedded GLB data, assignments, task values, baseline dates, warnings and unresolved relationship records travel with it. **Open project** validates before opening and asks before replacing the workspace.

Committed edits also attempt a local recovery copy. Recovery storage has a quota; a large embedded model may exceed it. The status changes and an error message instructs you to save a file. A successful recovery write is not proof that you saved a portable project copy.

Undo/Redo retain up to 40 full project snapshots in memory. Opening or importing a schedule clears that history. Large embedded models increase memory use. Task deletion removes its incoming/outgoing links and unassigns affected model parts; Undo can restore them.

Export schedule creates a CSV of selected task fields. Dependencies, geometry, baselines and import provenance are not included; use `.struxelo` for a complete supported project. Spreadsheet formula prefixes are escaped in CSV text cells.

Keyboard shortcuts outside text fields: Ctrl+Z Undo, Ctrl+Y or Ctrl+Shift+Z Redo, Ctrl+S Save, and Space Play/Pause. The in-app guide and property labels remain available without a network connection.

Data formats and interoperability

Struxelo project schema 1

A `.struxelo` file is UTF-8 JSON. It can also be opened with a `.json` extension. Core structure:

```
{
  "schema": 1,
  "name": "Example project",
  "tasks": [{
    "id": "A100", "name": "Foundations", "wbs": "Civil", "code": "FOUND",
    "start": "2026-11-01", "finish": "2026-11-12",
    "progress": 25, "budget": 120000, "actualCost": 28000,
    "predecessors": [], "summary": false, "critical": false
  }],
  "parts": [{
    "id": "part-1", "name": "FOUND / Raft", "taskId": "A100",
    "position": [0, 0, 0], "size": [10, 0.5, 8]
  }],
  "baseline": {"A100": {"start": "2026-11-01", "finish": "2026-11-12"}},
  "modelAsset": null,
  "warnings": [],
  "unresolvedDependencies": []
}
```

`taskId: null` means unassigned. Each task and part needs a unique string ID. Progress is 0–100. Costs are finite, nonnegative numbers. Dates must be valid ISO calendar dates with finish on or after start. Dependency records are `{ "id": "A090", "type": "FS", "lag": 0 }`; they refer to predecessor task IDs.

`baseline` is optional/null or a task-ID-keyed map of date pairs. `modelAsset` is null for box geometry, or a base64 data URL containing the complete GLB. When GLB data exists, mesh traversal IDs define the connection to `parts`; the part boxes are bounds metadata, not decoded mesh replacements.

Prototype limits are 10,000 tasks, 20,000 parts, a 25 MB GLB, a 30 MB schedule input and a 40 MB desktop project read. These are admission limits, not performance guarantees. Imported GLB data expands in JSON and Undo snapshots.

Microsoft Project XML subset

The importer recognizes the XML Project/Tasks structure, removing XML namespace prefixes while preserving field text. It skips the UID 0 project root and null task placeholders. It retains non-root summary rows.

Source	Internal field
`UID`, `ID`, `Name`, `WBS`	`id`, `code`, `name`, `wbs`
`Start`, `Finish`	Whole-date `start`, `finish`
`PercentComplete`	`progress`
`Cost`, `ActualCost`	`budget`, `actualCost`
`Summary`, `Critical`	Source flags
`PredecessorLink`	Internal predecessor record

Relationship type mapping follows MSP's numeric FF/FS/SF/SS representation. `LinkLag` uses an 8-hour-day approximation: source tenths of minutes divided by 4,800. Working calendars, source duration formats, constraints, resource assignments and elapsed-time variants are not evaluated. An imported `Cost` field is mapped for convenience; equivalence to contractual BAC is not established.

Primavera P6 XML subset

The importer recognizes one `APIBusinessObjects/Project`. More than one project is rejected pending an explicit project-selection workflow. It reads activities, WBS rows and relationships.

Activity `ObjectId` is used to resolve links to the activity's external `Id`. Missing IDs are not guessed or stripped. Unsupported relationship types and missing endpoints are retained in `unresolvedDependencies`, with source endpoint IDs and lag hours. The import report gives their count. These records block recalculation.

Start/Finish use source `StartDate` / `FinishDate` when present, otherwise planned date fields. Only their date components are retained. Progress is imported from `PercentComplete`; the source percent-complete method is not reproduced. Resource assignment cost rollups, calendars, constraints, actual-date rules and P6 scheduling settings are outside the supported subset. Lag hours are divided by eight as an explicitly reported approximation.

The public P6 sample contains 95 activities and 158 relationships. Struxelo resolves 155 and preserves three unresolved records. The missing endpoint strings include `_8`, `_10` and `_11`; this is an observation of that sample, not a general P6 rule.

CSV

Required headers: `id,name,start,finish`. Optional: `wbs,code,progress,budget,actualCost`. Quoted commas, quoted newlines and escaped double quotes are accepted. Dependencies are not imported from CSV. Empty optional numeric values become zero; date validation still applies.

CSV export contains the nine supported task columns. It is not a lossless project export. Text starting with spreadsheet formula characters is prefixed with an apostrophe. Use the native JSON project for complete supported state.

GLB

The importer accepts binary glTF with embedded buffers and textures. External URLs are refused. Decoder-dependent compression, animated/skinned assets and original material fidelity are outside the supported workflow. The current renderer assigns uniform construction-status materials and treats model coordinates as given.

No native decoding of `.sp`, `.spx`, `.dwf`, `.dwfx`, `.rvt` or IFC is implemented. A ZIP-based DWFX container inventory does not establish geometry decoding. REA's rejection of an SP file does not imply that its data is corrupt; it means the selected REA operation does not support that format.

Architecture and implementation notes

Concept and product direction: **Junaid Quamar, Senior Planning Engineer**.

Boundaries

Struxelo's Windows package consists of Electron's main process, a sandboxed preload bridge and a bundled browser renderer. The renderer owns the in-memory project and requests only explicit file operations from the main process.

File	Responsibility
`desktop/main.cjs`	Window lifecycle, native dialogs, size-limited file reads, temporary-file publication for saves
`desktop/preload.cjs`	Two narrow IPC methods: `openProject` and `saveFile`
`src/project.js`	Project validation, XML/CSV import, date arithmetic, metrics, assignment preview, FS calendar-day engine
`src/viewer.js`	Three.js scene, camera, GLB loading, selection, status appearance, bounding-box screening
`src/main.js`	UI state, task inspector, schedule, dialogs, commands, local recovery and interaction history
`src/style.css`	Desktop layout, responsive density, focus styles and brand tokens
`scripts/package-windows.mjs`	Pinned official Electron runtime acquisition, package assembly and ZIP output

The application does not import REA into its runtime. REA was used externally for the research phase and remains independently runnable. This distinction avoids implying that the app can decompile or natively open unsupported SYNCHRO projects.

Renderer lifecycle

`project` contains persistent data. `selectedTask`, `selectedPart`, focus date, current panel, playback state and filters are UI state. They are not part of the portable project schema.

Edits are made on a JSON clone. Validation runs before the new project is committed. A successful commit pushes the previous project into Undo history, clears Redo, updates the project, attempts local recovery and redraws the dependent panels. Opening a project or importing a schedule resets history and geometry state.

Three.js uses a persistent canvas and scene rather than reconstructing WebGL on each task change. Ordinary progress or assignment edits recolor existing meshes. Replacing geometry or applying a geometry-related Undo reloads the model. A generation counter prevents an older asynchronous load from becoming the current displayed model.

Mesh geometries, display materials and referenced textures are disposed when the old scene is cleared. Imported models receive standardized construction-status materials. Camera controls use damping and a ResizeObserver keeps the canvas aligned with its container. The renderer uses at most device pixel ratio 2 to limit high-DPI cost; large geometry can still consume substantial GPU memory.

Dates, dependencies and calculations

Dates are integers representing UTC calendar days. Display formatting explicitly uses UTC. This avoids daylight-saving shifts in whole-day calculations; it also means intra-day schedule semantics are intentionally lost during import.

Dependency records retain FS, SS, FF and SF types. Validation checks missing endpoints, self-links and cycles. The recalculation engine accepts only flat FS networks. It performs a topological forward pass for earliest dates and a reverse pass for latest dates and total float. Activity duration is its current finish-minus-start date span, and predecessor lag is measured in calendar days.

The engine does not solve calendar exceptions, maximum lags, external relationships, resource leveling, imposed constraints or source scheduling options. Retained unresolved P6 links block the UI's recalculation command. Showing imported source critical flags does not establish a newly computed critical path.

Earned value is a prototype physical-percent calculation. Planned value is linear across baseline dates. Task summaries are excluded from totals. Zero denominators produce an unavailable ratio, not infinity. Detailed formulas are in the user guide.

Geometry and assignments

Demonstration geometry consists of independently authored boxes. GLB import creates one resource per mesh, identified by stable traversal index (`mesh-0` , `mesh-1` , and so on). The entire embedded GLB is saved with the project; it is reparsed on reload in the same traversal order.

Assignments map one part to one task. This is simpler than SYNCHRO's full resource/appearance model. Automatic assignment only proposes unique code-substring matches, preserving existing assignments. An explicit preview precedes writes.

Overlap screening derives world-space axis-aligned bounds. It tests pairs of parts whose tasks are active on the focus date. It is $O(n^2)$, limited to 1,000 active parts, and may report false positives for rotated or concave geometry. It does not detect continuous-time collisions between frames.

Persistence and desktop isolation

The renderer has no Node.js APIs. Electron enables `contextIsolation` , `sandbox` and `webSecurity` , disables `nodeIntegration` , denies new windows and blocks page navigation. A content security policy limits executable content to local assets and prevents arbitrary object embeds. The preload bridge exposes no generic file path or process execution API.

Native Open reads only the user-selected file and limits its size to 40 MB. Native Save accepts a limited set of export kinds and writes through a sibling temporary file before rename. This reduces partial-file exposure but is not a transaction history or a guarantee against filesystem faults. Local recovery uses localStorage and can fail due to quota or access restrictions.

GLB resource URL rewriting permits only `data:` and `blob:` embedded resources. External URI fetching is refused. DTDs and entity declarations are rejected before XML parsing. UI values are HTML-escaped before insertion. CSV export escapes potentially executable spreadsheet prefixes. These controls do not replace testing of malformed, very large or adversarial inputs.

Build and distribution

Vite bundles the renderer into relative assets so Electron can open `dist/index.html` using `file:` URLs. The desktop shell uses only Electron and Node built-ins; runtime npm installation is unnecessary.

The Windows packaging script obtains Electron 45.0.0 using @electron/get's checksum verification, customizes its executable icon and product metadata, installs the built app under `resources/app` , and copies manuals and notices. It produces an unsigned portable ZIP. A native Windows build agent, signing identity, installer and update policy are future release work.

Next architectural work

Before expanding to large commercial projects, separate command handlers from DOM rendering, move long parsing/geometry work into workers, replace full-snapshot history with bounded commands, and add a documented calendar-aware scheduling engine. Introduce versioned migrations before changing the saved schema. Source-application compatibility needs exact fixtures, comparison rules and an observed SYNCHRO baseline.

Research log and evidence

Research date: **26 October 2026**. Product direction: **Junaid Quamar**.

Method

Separate three kinds of evidence:

1. **Documented:** a vendor page describes a feature.
2. **Observed:** a downloaded artifact was inspected or an implemented workflow was exercised.
3. **Unverified:** a behavior has not been compared against the reference desktop program.

No public document is treated as proof of implementation details, exact algorithms or complete parity. Struxelo is independently implemented; it does not contain recovered SYNCHRO source code.

Software acquisition

Bentley's release index lists 2026 releases including 6.5.11 and 6.5.10. The linked 6.5.11 article redirected to authentication in the research tool. The 6.5.10 announcement was readable and links to the official SYNCHRO download portal.

The download portal exposed a JavaScript loading shell, not a direct installer. Bentley's getting-started instructions direct users to sign into CONNECT and obtain download access. Its trial page uses a request-and-eligibility process. No account was created, no representative contacted and no proprietary installer was obtained.

Therefore a specific SYNCHRO executable version is not an observed baseline. Features in general documentation may differ across desktop releases, cloud offerings and license bundles.

REA installation and execution

REA source: https://github.com/morluto/rea
Commit: `0211a07ec5c43b2f8cbdbcd80f3f3619773a7b72`
Package: 6.8.0; Node: 24.22.1; npm: 11.16.0.

The source build succeeded. A production stdio adapter test initialized the server, listed 229 tools and six prompts, matched package/catalog identity, and completed a target-free `binary_session` request. This checks the adapter, not an active chat registration or native decompiler availability.

Public training material

Bentley's Training / Challenge Models page exposed a downloadable training ZIP.

Archive SHA-256:

```
40f5135cc898bc1d6892e11d9007954b836f8e879519c8cf0f77f11891aad749
```

REA `inspect-artifact` inventoried 16 files and three directories. Files include P6/MSP XML schedules, 3D DWF/DWFX assets and two SP projects. Complete evidence and the summary are under `research/evidence/`. The original inventory recorded the project's earlier working-directory name, `/workspace/veydris-4d`; files were subsequently moved to `/workspace/struxelo-4d`. Original evidence bytes were preserved.

The MSP sample contains 119 task elements; Struxelo imports 118 after excluding the UID 0 root. The P6 sample has 95 activities and 158 relationship elements. Static XML examination found three links with absent endpoints. Struxelo retains those records explicitly and reports that

only 155 relationships were resolved.

REA also inventoried `Structure.dwfx` as an archive. This establishes container entries, not geometry semantics. REA rejected `SYNCHRO Training Building Completed.sp` with `unsupported_target` / `file_format_rejected`; the error is retained as `sp-inspection.json`. No SP decoder was inferred from the filename or binary prefix.

Documentation baseline

- Official feature overview: model/schedule/resource workflows, baseline comparison, costs, simulations and collaboration.
- SYNCHRO 4D Pro guide: workflow topics and links to detailed help.
- Importing schedules: schedule product and format context.
- Auto Matching: attribute-based resource/task matching and preview/apply workflow.
- System requirements: Windows x64 reference-product requirements.

The feature matrix links requirements to these sources. Documentation-derived rows remain unverified until tested against an exact reference build.

What would establish parity

Obtain an authorized SYNCHRO executable, record its exact version and hashes, and build paired fixtures for schedules, calendars, constraints, model IDs, resources, appearance profiles, progress and costs. Compare imports, recalculated dates, values, visible geometry and exports under the same inputs. Record numerical tolerances and rendering differences. Expand the coverage ledger for each supported capability and retain failures; a visually similar screen is not evidence of semantic equivalence.

Feature matrix

This is a capability inventory, not a parity percentage. “Implemented” means a limited independent workflow exists; it does not mean equivalence to SYNCHRO has been established.

Vendor baseline: official feature overview, guide and schedule imports.

Capability	Struxelo 0.1.0	Boundary / missing work
Schedule authoring	Implemented subset	Flat edits; no complete planning engine
P6 XML import	Partial	95 sample activities; three unresolved links retained
MSP XML import	Partial	118 non-root sample tasks; calendars/constraints omitted
Other schedule integration	Missing	No Asta, Safran, NetPoint or two-way sync
CPM calculation	Partial	Flat FS network; calendar-day approximation
Baseline dates	Implemented subset	One date snapshot; no historical revisions
Activity filtering/sorting	Implemented subset	Text search, date/ID/source order
Multi-format geometry	Partial	Embedded GLB; no native CAD or SP decoder
Model updates	Partial	Replace whole model; assignments reset
Primitive geometry	Demo only	No interactive authoring tools
Construction subdivision	Missing	No geometry splitting or quantity takeoff
Manual linking	Partial	One task per part
Automatic linking	Partial	Unique code-substring preview
Resource planning	Missing	No full hierarchy, leveling or equipment allocation
Sequence playback	Implemented subset	Whole-day dates and three rates
Appearance simulation	Partial	Standard status colors; no full appearance profiles
Progress recording	Partial	Entered task percentage; no resource status history
Cost / earned value	Partial	Entered costs and physical-percent EV
Spatial analysis	Approximation	Active AABB screen, not solid clash detection
Reports/export	Partial	Task CSV and viewport PNG
Animation/rendering	Missing	No video, camera paths or Iray
Custom fields/formulas	Missing	No formula or script language
Cloud collaboration	Missing	Local single-user project only
Field / modeler integration	Missing	No mobile or cloud product equivalent
Windows distribution	Packaged prototype	Unsigned; requires native acceptance testing

Full feature accuracy and 100% parity remain **unverified and unmet**. There is no defensible completion percentage without an agreed version-specific capability inventory and observed comparisons.

Review-informed additions—guided onboarding, readable date stepping, local recovery and previewed matching—are design decisions for Struxelo. They do not establish that the reference application lacks those features.

Public feedback and product decisions

Product direction: **Junaid Quamar, Senior Planning Engineer**.

Source quality

Feedback was read from direct review pages and an indexed vendor idea. Review dates, incentives, product identity and context matter. Older complaints do not establish current defects, and contradictory experiences should not be flattened into a single verdict.

The Capterra SYNCHRO reviews include a 2018 account of feature overload, a 2022 comment on model-transfer materials/equipment movement and a 2025 concern about cost for smaller companies. These are qualitative observations, not representative statistics.

The G2 SYNCHRO Construction reviews include comments from 2019–2020 about interface discoverability, resource-tree learning and schedule synchronization setup. Some were incentivized. One review discusses traffic-signal modeling, which appears to concern a different SYNCHRO product; it was excluded from construction-planning decisions.

An indexed vendor idea requests more playback controls. Opening the original page returned 404 during research. Its indexed description is a weaker signal than a readable full discussion; it is not treated as evidence of current behavior.

Decisions

Signal	Assessment	Implemented response	Validation
Features can be hard to discover	Logical for a dense planning interface	Visible three-step workflow, Guide and contextual task inspector	Renderer workflow test; user acceptance still needed
Resource relationships can be difficult to understand	Logical for model/schedule linking	Searchable parts, explicit assignment count, selected-part linking, unique-match preview	Manual linking and preview smoke checks
Playback precision is useful	Logical for reviewing a construction sequence	One-day stepping, date scrubber, start/end controls and selectable rates	Playback/step smoke checks
Synchronization setup is difficult	Useful interoperability concern	Preserve imported dates and display omissions; retain unresolved IDs	Real P6/MSP fixtures and refusal checks
Material-transfer and equipment-motion fidelity matter	Valid but requires format/runtime evidence	Recorded as a gap; no claim of solving it through GLB status coloring	Not implemented
Cost is a barrier	Product consideration, not a code bug	No invented pricing model or licensing claims	No pricing decision made

Local recovery, Undo/Redo and explicit import confirmation were added as original usability decisions. They are not attributed to a review that did not request them.

UX principles used

Give the model and schedule most of the screen. Keep properties next to selection. Use color consistently for progress and a different color for selection. Place destructive or replacing actions behind concrete previews. Keep calculation assumptions visible at the decision point. Use readable labels and keyboard focus states. Treat recovery as fallible and give users a portable save option.

Next usability study

Ask planners to import a schedule, link five components, identify an unassigned component, review a date, record progress, compare baseline dates and save/reopen the project. Measure task completion, mistaken assignments and points where users seek help. Test both 1100 × 800 and

larger desktop layouts. Validate with actual planning engineers before calling the interface production-ready.

Brand identity and credit

Approved name: **Struxelo 4D**.
Concept and product direction: **Junaid Quamar, Senior Planning Engineer**.

Junaid supplied the navy-and-amber Struxelo 4D brand sheet. Application assets were prepared from that visual reference using image extraction. They are raster adaptations, not a claim to possess the original vector artwork.

- `public/brand/struxelo-wordmark.png`: navy horizontal wordmark for light backgrounds.
- `public/brand/struxelo-icon.png`: navy rounded-square 4D icon.
- `public/brand/struxelo.ico`: Windows icon format with multiple sizes derived from the icon asset.

The wordmark appears in the workspace header, About panel and documentation cover. The square mark is used for the browser favicon, desktop window icon and packaged executable resource.

Use navy as the main navigation and action color, amber for brand accents, teal for completed work and blue for selection. Progress colors must also be described in text so color is not the only signal. Do not distort the logo or place the navy wordmark on a navy background.

Keep the credit line in About and the documentation. Contact details from the supplied CV are not published. Third-party brands and source material retain their own ownership. Preliminary web naming research is not a legal clearance or a registered trademark claim.

Validation and release acceptance

Executed checks

The development environment is Linux. Passing renderer tests does not establish successful execution of the Windows shell.

Check	Result / evidence
REA source build	Passed; 6.8.0 from recorded GitHub commit
REA stdio adapter	Passed; 229 tools, six prompts, matching identity and `binary_session` request
Public training ZIP inspection	Passed; REA inventory and SHA-256 retained
SP inspection	Unsupported target, recorded as a limitation
Schedule/data unit tests	11 passed after fixing import handling and correcting the metric fixture's baseline
Browser interaction test	Passed 13 workflow checks; `research/ui-validation.json`
Model/file interaction test	Passed six checks; `research/model-validation.json`: GLB, assignment, embedded project save/reload, PNG and reopen
UI screenshots	`research/screenshots/` records large and compact desktop renderer layouts
Production renderer build	Vite build exercised; packaged assets generated locally
Windows runtime packaging	Built x64 portable ZIP; parsed executable verifies Struxelo name, credit, version and six icon sizes. Native execution remains untested

Unit-test scope

The suite covers JSON round-trip validation, strict date handling, branching FS critical-path/float calculation, cycle refusal, unsupported recalculation refusal, dangling assignments, negative costs, duplicate IDs, ambiguous matching, physical-percent earned value, summary exclusion, quoted CSV and spreadsheet-prefix escaping, malformed/DTD XML, multi-project refusal and both official XML samples.

The P6 fixture checks 95 activities, 155 resolved relationships and three retained unresolved links. MSP checks 118 non-root task rows. These checks validate the implemented subset; they do not compare with results from a running source application.

Renderer test scope

`tests/ui-smoke.mjs` runs Chromium against the local dev server. It checks branding, WebGL startup, task editing, Undo/Redo, focus-date stepping, playback, searching, assignment previews, the Costs panel, credit display, compact layout, real XML imports, unresolved-link recalculation refusal, reload recovery and invalid-project refusal. It captures console errors and writes the result to a JSON report.

Run with `CHROMIUM_PATH` if Chromium is elsewhere, and `STRUXELO_URL` if the dev server uses another address. The tests use temporary browser profiles; they do not open the user's real project profile.

Required Windows acceptance

- [ ] Extract the complete ZIP on Windows 10/11 x64.
- [ ] Start the executable and verify renderer, logo, window controls and task selection.
- [ ] Open and save through native dialogs, including cancellation.
- [ ] Save over an existing file; reopen and compare supported state.

- [ ] Test paths with spaces, Unicode names and non-writable destinations.
- [ ] Import both official XML examples and inspect warnings.
- [ ] Import a supported self-contained GLB; save/reopen and verify assignments.
- [ ] Test GPU behavior on representative workstations.
- [ ] Confirm recovery after ordinary close and unexpected termination.
- [ ] Check keyboard-only navigation, 125%/150% scaling and high-contrast needs.
- [ ] Verify package hashes and third-party license presence.
- [ ] Review unsigned executable behavior and decide on a signing/installer policy.

Required semantic comparisons

Before claiming SYNCHRO parity, choose an exact executable version and compare deterministic fixtures for calendars, FS/SS/FF/SF logic, lags, constraints, milestones, status dates, actual dates, floats, resource loading, costs, model identifiers, appearance profiles and exports. Record missing coverage instead of converting it to a pass.

Known release risks

The prototype uses full-project Undo snapshots and localStorage recovery, which are expensive for large embedded models. The UI is synchronous for some large data operations. Scheduling deliberately omits working calendars and source scheduling options. The Windows package is unsigned and not yet validated on Windows. These are explicit release limits, not hidden equivalence assumptions.

Credits

Concept and product direction

Junaid Quamar
Senior Planning Engineer

Junaid requested and directed the independent construction planning application, selected Windows desktop as the target platform, approved the Struxelo 4D name, requested improvements informed by public user feedback, and supplied the brand identity.

The title above was confirmed from the user-provided CV. The application and public documentation do not include the CV, contact information, employment history or other personal details.

Software implementation

Implementation, research organization and documentation were assisted by **OpenAI Codex**, following Junaid's direction. This is a factual project credit; it does not assert that Junaid authored third-party software or personally validated every calculation.

Research tooling and references

- **REA**, by morluto and its contributors: https://github.com/morluto/rea — MIT license. Installed separately in `/workspace/rea`; used for static artifact inventories and evidence.
- **Bentley Systems**: public SYNCHRO documentation and training archive, used as references. SYNCHRO names and third-party materials retain their original ownership.
- Direct user feedback on Capterra and G2, used as product research signals with source and age recorded.

No Bentley logo, proprietary executable or Bentley sample model is embedded into Struxelo's demonstration building. The demonstration geometry and schedule were created independently for this prototype. The research workspace keeps downloaded reference materials separate from the Windows application package.

Third-party notices

Struxelo 4D uses the following components. Attribution here does not replace their full license texts.

Component	Use	License / reference
Electron 45.0.0	Windows shell and Chromium runtime	MIT; https://github.com/electron/electron
Three.js 0.186.1	3D rendering, GLB import and orbit navigation	MIT; https://github.com/mrdoob/three.js
fast-xml-parser 5.5.8	Schedule XML parser	MIT; https://github.com/NaturalIntelligence/fast-xml-parser
Vite 8.3.5	Development server and build tooling	MIT; https://github.com/vitejs/vite
@electron/get 3.1.0	Build-time runtime acquisition and checksum verification	MIT; https://github.com/electron/get

The Windows ZIP retains Electron's `LICENSE` and `LICENSES.chromium.html`. Renderer dependency license texts are copied into the distribution's `third-party-licenses` directory. `package-lock.json` records exact installed dependency versions and integrity hashes for source builds.

REA is separately installed tooling; its source and MIT license remain in the REA checkout. It is not bundled into the Windows application. Public Bentley materials are stored in the research workspace and are not redistributed as part of the application. Struxelo is not affiliated with or endorsed by Bentley Systems. No blanket license is granted here for third-party reference material or the supplied logo.